

Radar Search Pipeline AS IS

Status: AS IS

Product area: Radar candidate and signal search

Updated after slice: 0.7.6.3.5

Last updated: 2026-06-27

Canonical source: current implementation, tests, ROADMAP.md, and Radar run diagnostics

Generated PDF: docs/radar/RADAR_SEARCH_PIPELINE_AS_IS.pdf

1. Purpose

This document explains the current Radar search pipeline as it is implemented today. It is the operational map for candidate discovery, source handling, identity enrichment, signal search, diagnostic states, and benchmark evaluation.

The document has two jobs:

- help a user or new developer understand how Radar search currently works;
- help agents and developers find the correct extension point before changing planner, retrieval, extraction, registry lookup, checkpoints, signal search, dossier projection, or evaluation.

The Markdown file is the source of truth. The PDF is a generated review artifact.

Mermaid source can remain in Markdown, but the PDF must contain rendered diagrams instead of raw diagram notation.

2. AS IS And TO BE Rule

There is exactly one current AS IS document:

- docs/radar/RADAR_SEARCH_PIPELINE_AS_IS.md
- docs/radar/RADAR_SEARCH_PIPELINE_AS_IS.pdf

Substantial Radar pipeline changes must start with a TO BE design:

- docs/radar/to-be/RADAR_SEARCH_PIPELINE_TO_BE_<slice>.md

The TO BE document describes the intended algorithmic change before implementation. After implementation and validation, the accepted behavior is merged into this AS IS document and the PDF is regenerated.

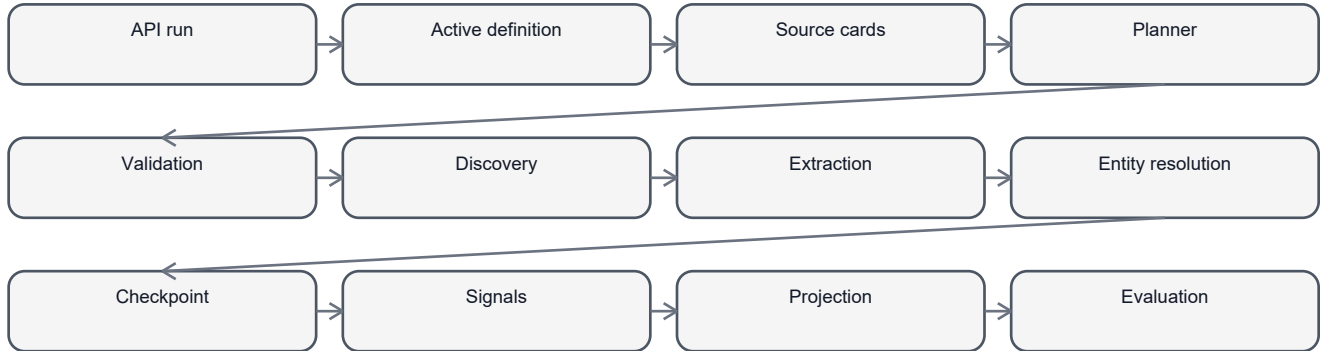
3. Glossary

Term Meaning
--- ---
Radar definition Persisted active configuration for one Radar, including criteria, signals, source policy, and scoring settings.
Candidate universe The broad set of source-backed entities known to the run, including legal entities and review-needed sites, branches, projects, or assets.
Product candidate A strict account candidate shown as a scored product row. Product candidates should be legal entities or resolved account-level entities.
Review-needed entity A source-backed entity retained upstream with review flags, but not promoted as a confident product account.
Signal A monitored buying-intent or activity indicator evaluated for candidate entities.
Source A user-selected information source in Radar settings.
Connector profile Human-readable config describing a connector, good inputs, bad inputs, expected facts, limitations, and credentials.
Capability card Backend-compiled machine-readable connector capability used by preflight, planner input, and execution guards.
Planner source card Compact planner-facing source description compiled from capability, source definition, and source obligation.
Source obligation User-facing source usage mode such as required_for_identity, required_for_coverage, preferred, fallback, or disabled.
Retrieved source URL/snippet/citation/source material returned by retrieval before extraction.
Analyzed source Source material inspected by extraction or diagnostics but not necessarily linked to evidence.

Used source	Evidence-bearing source linked to a product candidate or finding.
Evidence ref	Stable source reference used to link extracted candidate/finding rows to normalized source records.
Checkpoint	Application-owned review point that decides whether execution continues, retries, expands, revises, stops for review, or fails hard.
Execution budget	Radar task budget for discovery, gate, signal, provider, and run-total work.
External-call budget	Budget for external actions such as OpenRouter calls, DaData lookups, provider retries, and source verification requests.
not_observed	A searched signal with no evidence found. It must not mean "not searched".
not_searched_*	Explicit unsearched state caused by budget, policy, missing scope, or pending output.

4. High-Level Pipeline

Rendered diagram: high-level Radar pipeline



The pipeline is application-owned. Integrations execute bounded provider tasks, but they do not own scoring, review semantics, source obligations, or final candidate state.

5. Backend Roles

| Role | Current owner | Responsibility | Must not own |
 |---|---|---|
 | API route | src/power_web_os/api | Create runs, expose status, candidates, dossier, runtime/preflight DTOs. | Provider calls, scoring, SQL queries in routes. |
 | Worker endpoint | src/power_web_os/jobs | Load run id, call application executor, persist status. | Provider normalization or domain decisions. |
 | Workflow wrapper | src/power_web_os/workflows | Wrap application execution in workflow state when needed. | SQLAlchemy queries, provider logic, scoring semantics. |
 | Active definition adapter | src/power_web_os/application/live_radar_definition_runtime.py | Convert persisted definition payload into runtime Radar payload. | HTTP/provider details. |
 | Connector profile registry | src/power_web_os/application/connector_profiles.py | Load connector profiles and compile capability cards. | User source obligations or provider calls. |
 | Planner input builder | src/power_web_os/application/live_radar_discovery_planning.py | Build source-card-aware planning input and deterministic fallback plans. | Final truth or provider execution. |
 | Planner adapter | src/power_web_os/integrations/openrouter_discovery_planner.py | Ask OpenRouter for structured discovery plans. | Source policy enforcement. |
 | Plan acceptance | src/power_web_os/application/live_radar_plan_acceptance.py | Validate capabilities, source obligations, and safe repairs. | Provider calls. |
 | Retrieval plan compiler | src/power_web_os/application/live_radar_retrieval_plan.py | Convert accepted plan steps into bounded provider task cards. | Scoring. |
 | Web retrieval/extraction provider | src/power_web_os/integrations/live_radar_openrouter.py | Execute OpenRouter web/retrieval/extraction tasks under budget guard. | Execution budgets, final scoring, source obligations. |
 | Source registry/provider orchestration | src/power_web_os/application/radar_source_providers.py | Execute structured company registry providers for allowed stages. | Signal evidence replacement. |
 | Registry lookup term generator | src/power_web_os/application/radar_registry_lookup_terms.py | Build concrete lookup terms for registry providers. | Broad web discovery. |
 | Search expansion service | src/power_web_os/application/radar_search_expansion.py | Build bounded query variants when discovery/coverage is weak. | Direct provider calls. |
 | Extraction contract/repair | src/power_web_os/application/live_radar_extraction_contract.py | Validate and repair provider payload shape when deterministic repair is safe. | Silently converting unrecoverable output into success. |
 | Checkpoint service | src/power_web_os/application/live_radar_checkpoints.py | Decide continue, retry, expand, repair, revise, stop, or fail. | Direct HTTP/provider calls. |
 | Checkpoint action executor | src/power_web_os/application/live_radar_checkpoint_actions.py | Apply approved checkpoint actions under budgets and policy. | Unbounded loops. |
 | Entity resolution | src/power_web_os/application/live_radar_entity_resolution.py | Distinguish legal entity, branch, production site, project, asset, and unknown entity. | Provider transport. |
 | Candidate universe support | src/power_web_os/application/live_radar_universe.py and retrieved-candidate helpers | Preserve source-backed legal entities and review-needed upstream entities. | Product precision claims. |
 | Dossier projection | src/power_web_os/api/radar_dossier_mappers.py and related mappers | Explain lifecycle, diagnostics, checkpoints, budgets, candidates, and sources. | Mutating run behavior. |
 | Evaluation | src/power_web_os/radar_evaluation.py | Compare persisted run/dossier output to curated baseline. | Live provider calls. |

6. Inputs And Runtime Configuration

The live run is driven by these inputs:

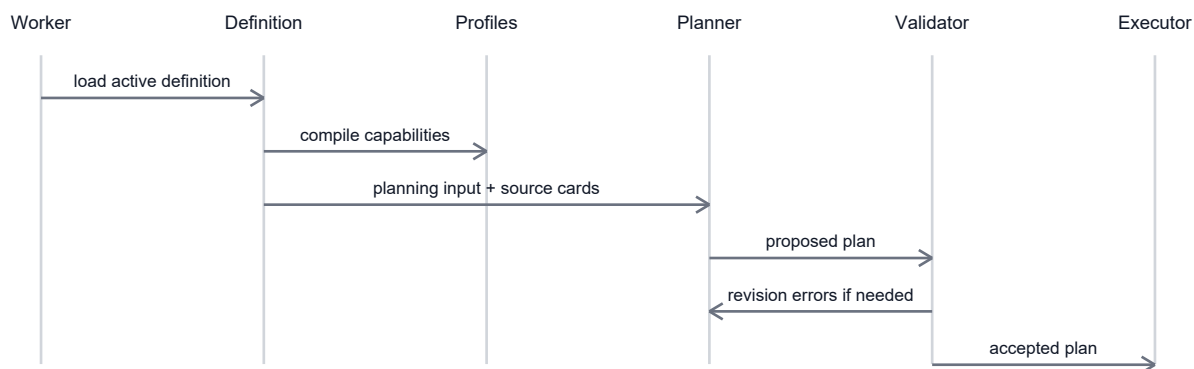
- active RadarDefinitionRecord loaded by radar_id;

- canonical runtime Radar payload built from the active definition;
- `global_search_policy.sources`;
- source usage obligations from Radar settings;
- connector profiles under `config/connectors`;
- compiled capability cards and planner source cards;
- runtime config from `.env` and task context;
- execution budgets and external-call budgets;
- run profile such as `live`, `smoke`, `benchmark_smoke`, or `benchmark_live`;
- persisted run metadata from API queue time and worker execution time.

Secrets stay in `.env` or deployment secret storage. They are never passed to planner source cards, dossier, journal, evaluation reports, or AS IS/TO BE docs.

7. Planning Loop

Rendered diagram: planner and validation sequence



Current planning behavior:

1. The worker loads the active persisted Radar definition. The hardcoded mini Radar is legacy/offline fallback only.
2. Connector profiles are compiled into source cards for configured sources.
3. Planner input includes source cards and source obligations. Source cards say what a source can do; obligations say how strongly the selected source must be used.
4. The planner may use OpenRouter and must return a structured plan.
5. Backend validation rejects incompatible source use, disabled sources, missing required source use, and lookup-only sources used for broad discovery.
6. Invalid plans can enter a bounded revision loop. If revision remains invalid, execution stops as review-needed or policy-blocked instead of falling into blind execution.

Planner calls count against OpenRouter external-call budgets, including total OpenRouter run budget and planner-role budget.

8. Source Cards And Connector Profiles

Connector profiles are human-facing config. They describe a connector in terms a

connector author can understand:

- display name;
- description;
- good inputs;
- bad inputs;
- expected facts;
- limitations;
- credential environment variable names;
- runtime provider id.

Application code compiles those profiles into capability cards:

- lookup-only or broad-discovery-capable;
- identity/enrichment/coverage/signal applicability;
- required input kinds;
- returned fact kinds;
- useful-result criteria;
- credential requirements.

Planner source cards are compact, product-safe versions of the capabilities.

They contain no credentials, API keys, headers, tokens, or provider secrets.

9. Retrieval, Extraction, And Recovery Loop

The retrieval/extraction path is split conceptually:

1. Retrieval obtains ranked sources, snippets, URLs, citations, and source outcomes.
2. Extraction maps task cards plus retrieved material into structured observations, candidates, source outcomes, coverage findings, and candidate universe gaps.
3. Schema validation checks provider output shape.
4. Deterministic repair attempts safe shape fixes.
5. If repair fails, one bounded primary retry can be attempted with strict JSON context.
6. If configured, one bounded backup extraction model retry can be attempted for extraction-stage tasks only.
7. If still invalid, the branch stops with an explicit diagnostic reason such as `primary_schema_invalid`, `backup_schema_invalid`, `backup_not_configured`, or `budget_exhausted_before_backup`.

Extraction recovery attempts are recorded in `extraction_recovery_records` and must count against external-call and provider-retry budgets.

10. Registry Lookup Loop

Company registry providers are structured identity/enrichment sources. DaData is the first implementation. Registry providers are not broad discovery engines and

must not replace web evidence for signals.

The current lookup flow:

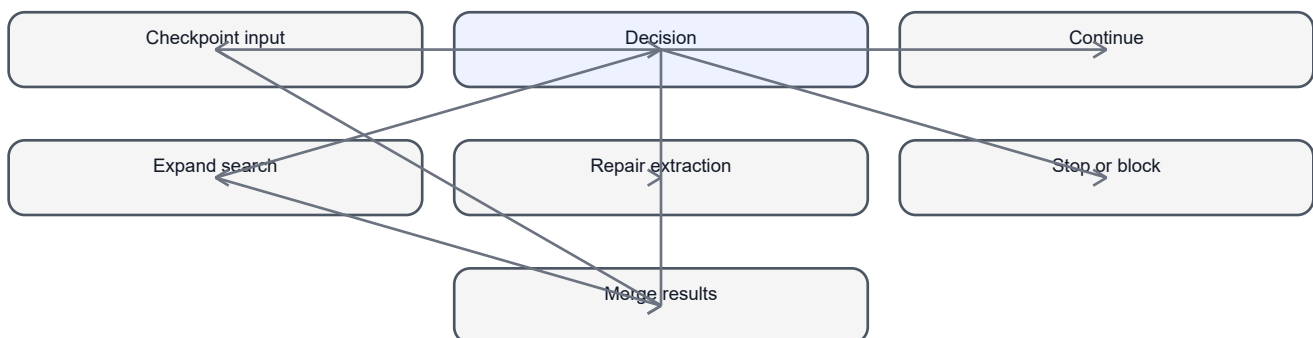
1. Build concrete lookup terms from candidate names, retrieved snippets, identifiers, Russian aliases, English aliases, legal-form variants, and short fragments.
2. Reject placeholders such as "candidates from step 1" and broad natural language discovery queries.
3. Try terms in bounded order: identifiers, Russian legal-form terms, Russian short names, then English aliases.
4. Count each lookup term against DaData lookup budget.
5. Stop after a useful match.
6. Preserve each lookup attempt in `registry_lookup_attempts`.

Important semantics:

- one alias `no_match` is not a hard block;
- `registry_lookup_insufficient` means there was no concrete registry input;
- `identity_not_confirmed_after_all_terms` means concrete terms were tried but identity still was not confirmed;
- ambiguous but source-backed entities can be retained as review-needed upstream entities or linked facts.

11. Search Expansion Loop

Rendered diagram: checkpoint loop



When discovery or coverage is weak, `RadarSearchExpansionService` can create bounded query variants:

- official-domain variants, for example source-specific site search;
- open-web relation variants;
- identity variants with INN/OGRN/legal-form hints;
- industrial/site variants with plant, GPP, site, branch, or asset language;
- alias/language variants from extracted or baseline-like diagnostic names.

Expansion respects source policy. If open web is disabled, open-web expansion is not generated. If an official source is disabled, official-domain variants are

not generated. Expansion must stay bounded by execution and external-call budgets.

12. Candidate Universe And Entity Resolution

The upstream candidate universe is recall-first. It can retain source-backed entities that are not yet strict product accounts.

Current rules:

- legal entities can become product candidates when evidence and resolution are sufficient;
- weak legal-entity mentions can remain review-needed universe entries;
- branches, factories, production sites, projects, and assets can be retained as review-needed upstream entities or linked facts;
- unresolved sites, projects, or assets must not become high-confidence product account candidates;
- review flags explain why an entity needs human attention.

Common review flags include:

- requires_human_review;
- not_standalone_legal_entity;
- registry_match_ambiguous;
- official_source_cross_checked;
- candidate_universe_gap.

Product candidate projection remains precision-first. Upstream universe retention is allowed to be broader than product account output.

13. Checkpoints And Adaptive Actions

The checkpoint service reviews execution after key stages:

- after discovery;
- after qualification gates;
- after coverage checks;
- before signal search.

Possible decisions:

- continue;
- retry_same_source;
- expand_sources or search expansion;
- repair_extraction;
- retry_extraction;
- revise_plan;
- stop_review_needed;
- fail_hard.

Actions are bounded:

- no unbounded retries;
- no policy bypass;
- no hidden broad fallback;
- no signal search until the pre-signal checkpoint allows it;
- all adaptive provider calls count against budgets.

If a stop is diagnostic rather than a runtime failure, the run can still produce a completed snapshot with `stopped_for_review_reason` and explicit checkpoint metadata.

14. Signal Search

Signal search is not candidate discovery. It runs only after candidate universe and qualification/coverage checkpoints allow it.

Rules:

- signal tasks should use web/official evidence sources, not registry enrichment sources unless a connector capability explicitly supports signal evidence;
- signal prompts include one signal and the current candidate scope;
- signal extraction must not add new scored candidates;
- new entities found during signal search go to candidate universe gaps or diagnostics;
- unsearched signals become `not_searched_*`, not `not_observed`;
- `not_observed` is reserved for searched-negative evidence or invalid searched evidence.

15. Budget Model

Radar has two complementary budget layers.

Execution budgets:

- discovery tasks per rule;
- gate tasks per candidate/rule;
- signal tasks per candidate/signal;
- provider task keys;
- total web tasks per run.

External-call budgets:

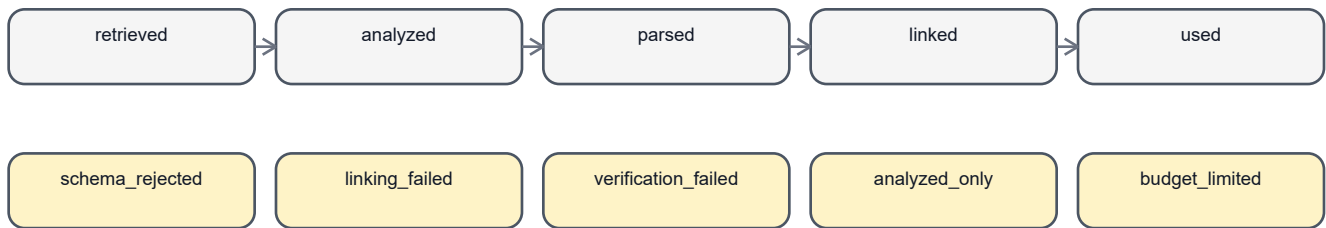
- total OpenRouter HTTP calls;
- OpenRouter planner calls;
- OpenRouter web task calls;
- OpenRouter server-tool web searches reported after the call;
- DaData/company registry lookups;
- source verification requests;
- provider retries per task.

Smoke and benchmark profiles set bounded task context budgets so acceptance runs do not spread into uncontrolled provider calls. OpenRouter latency alone is not

an error; the controllable unit is number of external actions.

16. Source Lifecycle

Rendered diagram: source lifecycle



Product source lists remain strict: only evidence-bearing used sources appear as product sources. Dossier/source lifecycle diagnostics are broader and must show retrieved, analyzed, rejected, unlinked, verification-limited, and budget-limited sources.

17. Dossier, Journal, Trace, And Evaluation

Radar exposes several diagnostic surfaces:

- run status and run metadata;
- dossier summary and source lifecycle;
- candidate universe and product candidates;
- checkpoint decisions and adaptive actions;
- source cards and capability validation;
- source obligation decisions and runtime outcomes;
- external-call budget counters;
- extraction recovery records;
- registry lookup terms and attempts;
- search expansion tasks and results;
- technical trace for sanitized developer inspection;
- benchmark and evaluation reports.

Forbidden everywhere:

- API keys;
- authorization headers;
- bearer tokens;
- raw prompts when not explicitly sanitized;
- raw hidden chain-of-thought;
- raw provider dumps that contain secrets or hidden reasoning keys.

18. Evaluation Loop

The benchmark/evaluation layer reads persisted run output and dossier data. It does not call OpenRouter, DaData, or source providers.

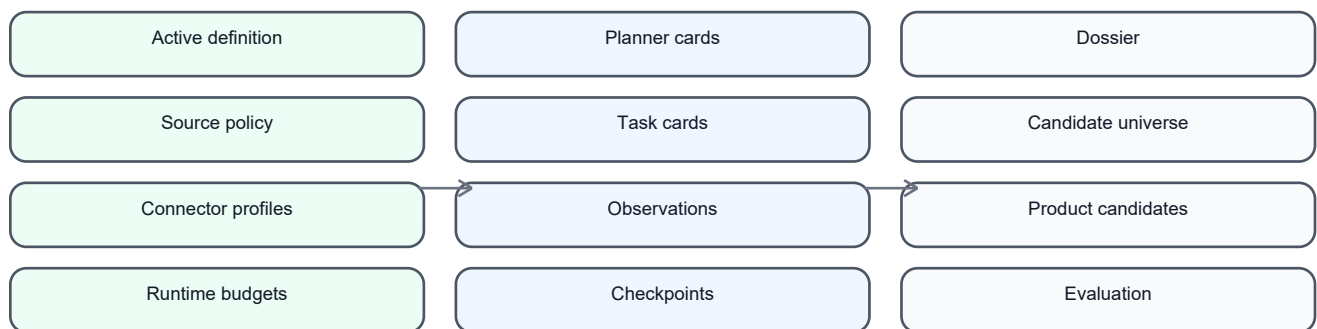
Current SIBUR evaluation channels:

- strict_recall for legal-entity baseline hits;
- review_recall for production-site, branch, asset, or project hits retained as review-needed universe entities or linked facts;
- precision for strict product account candidates;
- false positives;
- false negatives;
- ambiguous matches;
- evidence quality buckets;
- optional coverage probe output that is diagnostic only and does not change the original metrics.

Evaluation is a measurement layer. If it exposes poor quality, the fix should be planned as a follow-up slice rather than hidden inside the evaluation code.

19. Context Management

Rendered diagram: context and data-flow guard



Never pass secrets, raw hidden reasoning, or raw provider dumps.

```

| Receiver | Allowed context | Forbidden context |
|---|---|---|
| Planner | Radar goal, source cards, obligations, compact criteria/signals, budget hints. | Secrets, raw provider dumps, hidden reasoning, credentials. |
| Plan reviser | Product-safe checkpoint facts and capability validation errors. | Raw traces, hidden reasoning, secret-bearing payloads. |
| Web extractor | Task card, retrieved sources, expected schema, current candidate scope. | Full unrelated run history, secrets, scoring decisions. |
| Backup extractor | Failed extraction reason and strict task-specific JSON context. | Planner reasoning, broad source policy mutation. |
| Registry provider | Concrete legal names, INN, OGRN, legal-form variants, strong aliases. | Broad discovery queries and placeholders. |
| Checkpoint service | Counts, outcomes, warnings, budgets, source obligations, extraction issues. | Direct provider credentials or raw hidden reasoning. |
| Entity resolver | Provider observations, registry facts, source refs, task context. | HTTP client details. |
| Dossier mapper | Sanitized execution metadata and output snapshot. | Runtime side effects or scoring changes. |
| Evaluation | Persisted dossier/output and baseline fixture. | Live provider calls. |
  
```

20. Extension Points

```

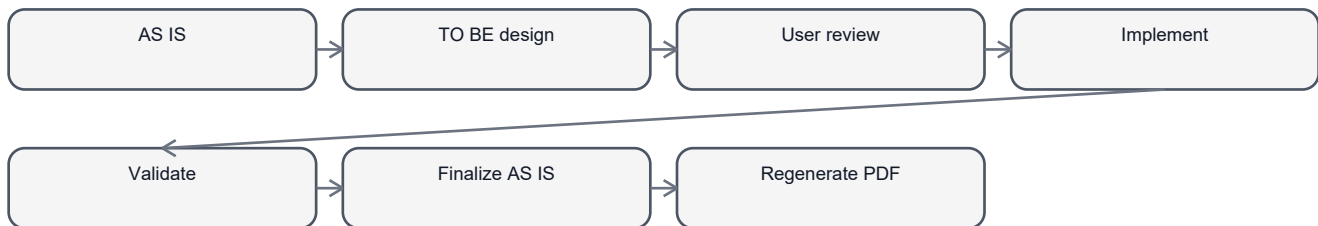
| Change | Extension point | Required validation |
|---|---|---|
| New source connector | config/connectors, connector profile registry, source provider port/adaptor. | Connector profile tests, preflight, source card validation. |
| Planner capability change | source card compiler, planning input, plan acceptance. | Planner/validator fake tests, no-secret trace tests. |
| Search expansion strategy | RadarSearchExpansionService. | Unit tests for query families, policy filtering, caps, dedupe. |
| Registry lookup terms | RegistryLookupTermGenerator. | Unit tests for aliases, Russian/English/legal-form terms, identifiers, placeholders. |
| Extraction schema repair | extraction contract and OpenRouter provider recovery path. | Malformed-output fixtures, retry/backup budget tests. |
| Entity resolution semantics | entity resolution and candidate universe projection. | Legal entity vs site/branch/project/asset tests. |
| Checkpoint policy | checkpoint service/action executor. | Recorded/fake adaptive pipeline tests. |
| Signal search behavior | retrieval task compiler and staged execution signal phase. | Not-searched vs not-observed tests. |
| Dossier projection | API dossier mappers and source lifecycle. | Backend API/dossier tests. |
| Benchmark metrics | evaluation module and baseline fixtures. | Evaluation unit/report tests. |
  
```

21. Test Map

Behavior	Typical tests
Active definition and runtime wiring	tests/test_radar_preflight.py, tests/test_persisted_live_radar.py
Connector profiles and source cards	tests/test_connector_profiles.py, tests/test_live_icp_radar.py
Source obligations	tests/test_live_icp_radar.py, tests/test_radar_preflight.py
External-call budgets	tests/test_radar_external_call_budget.py
Adaptive checkpoints	tests/test_radar_adaptive_execution.py
Search expansion and lookup terms	tests/test_radar_search_expansion.py, tests/test_live_icp_radar.py
Extraction recovery	tests/test_live_icp_radar.py, tests/test_radar_adaptive_execution.py
Dossier/API projection	tests/test_backend_api.py
Benchmark report	tests/test_radar_benchmark.py
Recall/precision evaluation	tests/test_radar_evaluation.py
Backend boundaries	tests/test_backend_architecture_contract.py

22. AS IS / TO BE Maintenance Lifecycle

Rendered diagram: AS IS / TO BE maintenance lifecycle



Required maintenance rules:

1. Before a substantial Radar pipeline change, create a TO BE document.
2. Generate or update the slice plan from that TO BE.
3. Implement the slice.
4. Run targeted tests and any required smoke/evaluation flow.
5. Update AS IS with implemented behavior.
6. Regenerate PDF.
7. Mark the roadmap slice Done only after AS IS/PDF are current.

23. Known Gaps

- This document is descriptive. It does not replace tests, dossier, trace, or benchmark reports.
- The AS IS document can drift if future slices skip the maintenance rule; the documentation contract test and slice acceptance criteria exist to reduce that risk.
- The PDF rendering command is intentionally project-local and may need tooling adjustment if the repository moves to another documentation stack.